Video Walkthrough — Shooting Script

AI Content Marketing Suite · magna-test-ten.vercel.app

Target length: 7–8 min (rubric allows 5–10). Format: screen recording + voiceover.

Must show (rubric): the text-generation flow, the image-generation flow, and a portion of the Claude Code workflow.

Prep before you hit record

- Open two browser tabs: (1) the live app https://magna-test-ten.vercel.app, (2) the GitHub repo.
- Open your editor/terminal with Claude Code in the project, and have `CLAUDE.md` open.
- Do one throwaway generation first so the model is warm and `/history` isn't empty.
- Record at 1080p+, hide bookmarks/notifications, zoom the browser to ~110%.

Each scene below has: [SHOW] what's on screen · [SAY] the voiceover · [EDIT] editing notes.

Scene 1 — Cold open (0:00–0:20)

- [SHOW] Land on the home page; let the self-running demo play for a few seconds (topic typing -> copy streaming -> image painting in).
- [SAY] "This is the AI Content Marketing Suite — a tool that writes marketing copy in four formats and generates a matching image for every post, all in one flow. Everything you're about to see is live. Let me show you."
- [EDIT] Title card overlay: *"AI Content Marketing Suite — Magna Labs 48h build"*. Keep it snappy.

Scene 2 — The 30-second tour (0:20–0:50)

- [SHOW] Scroll the landing page: the demo, the "How it works" steps, the "Under the hood" architecture section, the Core-vs-Bonus panel.
- [SAY] "The landing page doubles as onboarding — a live demo, a call-flow diagram of the whole system, and a clear split between the core assessment features and the bonus extras. Now let's use the real thing."
- [EDIT] Speed-ramp the scroll; hold ~1s on the architecture diagram.

Scene 3 — Text generation flow * required (0:50–2:40)

- [SHOW] Click Get started -> /create. Select Blog post. Type a topic (e.g. *"Why small teams should automate invoicing"*), set tone Witty, audience B2B SaaS founders.
- [SAY] "I pick a format, a topic, a tone, and an audience. Each of the four formats — blog, LinkedIn, ad, email — has its own prompt strategy on the server, not a shared template."
- [SHOW] Turn on the brand voice toggle ("Acme Co."). Click Generate content.
- [SAY] "I'll also apply a saved brand voice. Watch the copy — it streams in token by token as Claude writes it."
- [SHOW] Let the copy stream fully. Scroll it.
- [SAY] "That's real streaming from the server, not a spinner. The result is finished, on-brand copy."
- [SHOW] Go back, switch to Ad copy, generate again. Point at the three distinct variants.
- [SAY] "Different format, different strategy — ad copy always returns exactly three variants, each attacking a different angle. That count is enforced in the schema, not just requested in the prompt."
- [EDIT] Caption the streaming moment: *"live token streaming"*. Caption the ad result: *"3 variants — enforced in code"*.

Scene 4 — Image generation flow * required (2:40–3:55)

- [SHOW] On a generated post, click Generate matching image. Let the skeleton -> image paint in.

- [SAY] "One click builds an image. But it's not just the topic — an 'art director' step reads the *finished copy* and turns it into a concrete scene, so the picture actually reflects what the post says."
- [SHOW] Point at the auto-prompt caption under the image.
- [SAY] "You can see the auto-generated prompt right here."
- [SHOW] Click a different style pill (e.g. 3D render). Show the image change but the subject stay the same.
- [SAY] "Regenerate in a different style — same subject, new look — and download it. The image is re-hosted on permanent storage, so it never expires."
- [EDIT] Caption: *"content-aware prompt"* and *"6 styles, same subject"*. Consider a side-by-side of two styles.

Scene 5 — Improve, History & Export (3:55–5:05)

- [SHOW] Go to Improve, paste/reuse text, pick a goal (More persuasive), run it. Show the improved text + the "what changed" note.
- [SAY] "The improver refines existing text toward a goal — shorter, more persuasive, more formal, SEO, or re-audience — and explains what it changed."
- [SHOW] Go to History. Show the grid of saved cards with images. Open one (the modal). Open Download v -> PDF + photo; briefly show the downloaded file with the image embedded.
- [SAY] "Everything is saved to your session's history — view, copy, delete, and export any piece as text, Word, or PDF, with the image embedded."
- [EDIT] Caption: *"before -> after + what changed"* and *"export with embedded image (bonus)"*.

Scene 6 — Under the hood (5:05–5:50)

- [SHOW] Open /architecture (the "Architecture · optional" nav item). Scroll the design-choices / trade-offs. Then back to the landing "Under the hood" call-flow tabs — click Image to show the step-by-step function chain.
- [SAY] "Architecturally: all AI runs server-side behind REST routes — the browser never sees a key. Identity is an anonymous, HMAC-signed session cookie, and every read and write is scoped to that session. Images are re-hosted on Vercel Blob. It's all documented in the README and this in-app architecture note."
- [EDIT] Caption the call-flow: *"what calls what — server-side"*.

Scene 7 — Claude Code workflow * required (5:50–8:15)

This is the 15-point dimension — spend real time here, on camera.

- [SHOW] Open `CLAUDE.md` in the editor.
- [SAY] "I drove the whole build with Claude Code. This is my steering doc — the non-negotiables, the model config, the conventions, and the build order I followed."
- [SHOW] Open Claude Code and show a real interaction. Pick one or more of:
  - Planning: ask Claude Code to plan a feature and show the plan it produces.
  - Debugging (great story): scroll the git log / show the commits for the DALL·E -> Blob saga (`e18cc50` model fallback, `9ea492e` private-blob proxy) and narrate how Claude Code diagnosed the 502 and added the `/api/img` proxy.
  - Refactoring: show the declarative AI-error classifier (`src/lib/ai/errors.ts`) and say Claude Code generated + wired it across the routes.
  - AI-native QA: mention/show the multi-agent review + grading workflow you ran.
- [SAY] "Here's a real debugging loop — the image API was 502-ing; Claude Code traced it to a private Blob store and added a proxy route. And here it's refactoring error handling into one declarative classifier across every route."
- [SHOW] `git log --oneline` — scroll the clean conventional-commit history.
- [SAY] "The commit history mirrors that workflow: planned phases, a real debugging arc, and iterative polish."
- [EDIT] Zoom into code/terminal so it's readable. Keep this section concrete — show Claude Code *doing* something, don't just talk about it.

Scene 8 — Close (8:15–8:40)

- [SHOW] Back to the live app; show the URL bar and the GitHub repo tab.
- [SAY] "That's the full suite — multi-format generation, content-aware images, an improver, per-session history with export, and a brand voice — all server-side, deployed live on Vercel, built with Claude Code. Links to the live app and the repo are below. Thanks for watching."
- [EDIT] End card: live URL, repo URL, "Built with Claude Code".

Editing checklist

[ ] Text-generation flow shown end-to-end (Scene 3) '1
[ ] Image-generation flow shown end-to-end (Scene 4) '1
[ ] A real portion of the Claude Code workflow (Scene 7) '1
[ ] Captions on the key moments (streaming, content-aware image, export, server-side)
[ ] Readable code/terminal zoom in the Claude Code section
[ ] Under 10 minutes; intro and outro cards
[ ] Live URL + repo URL on screen at the end